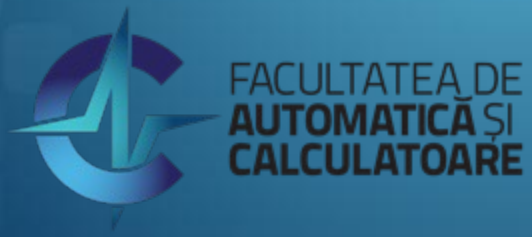


Algoritmi Paraleli și Distribuiți Java

Radu-Ioan CIOBANU
radu.ciobanu@upb.ro





Programarea concurentă în Java





APD

Java multithreading

Stările posibile ale unui thread

- Creat: obiectul a fost creat cu operația `new()`; se poate apela metoda `start()`
- Gata de execuție: a fost apelată metoda `start()`, firul poate fi executat
- Suspendat: a fost apelat `sleep()` sau `wait()`
- Terminat: metoda `run()` a fost terminată

Java multithreading

```
public class MyThread extends Thread {
	public void run() {
		System.out.println("Hello world!");
	}
}
```

```
public class Main {
	public static void main(String[] args) {
		int N = 4;
		MyThread threads[] = new MyThread[N];

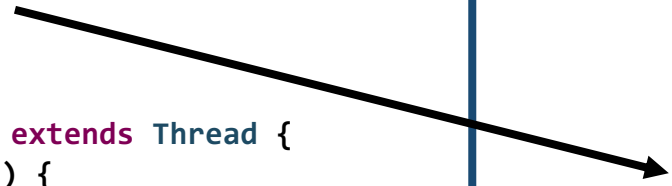
		for (int i = 0; i < N; i++) {
			threads[i] = new MyThread();
		}
		for (int i = 0; i < N; i++) {
			threads[i].start();
		}

		System.out.println("Hello from main thread");

		for (int i = 0; i < N; i++) {
			try {
				threads[i].join();
			} catch (InterruptedException e) {
				e.printStackTrace();
			}
		}
	}
}
```

start method

Thread-urile sunt pornite prin apelul metodei **start**() dintr-un obiect Thread



```
public class MyThread extends Thread {
      public void run() {
            System.out.println("Hello world!");
      }
}
```

```
public class Main {
      public static void main(String[] args) {
            int N = 4;
            MyThread threads[] = new MyThread[N];

            for (int i = 0; i < N; i++) {
                  threads[i] = new MyThread();
            }
            for (int i = 0; i < N; i++) {
                  threads[i].start();
            }

            System.out.println("Hello from main thread");

            for (int i = 0; i < N; i++) {
                  try {
                        threads[i].join();
                  } catch (InterruptedException e) {
                        e.printStackTrace();
                  }
            }
      }
}
```

start method

```
public class MyThread extends Thread {
      public void run() {
            System.out.println("Hello world!");
      }
}
```

```
void start() {
      Crează stack-ul pentru thread
      Informează JVM și OS
      Thread-ul execută metoda run();
}
```

```
public class Main {
      public static void main(String[] args) {
            int N = 4;
            MyThread threads[] = new MyThread[N];

            for (int i = 0; i < N; i++) {
                  threads[i] = new MyThread();
            }
            for (int i = 0; i < N; i++) {
                  threads[i].start();
            }

            System.out.println("Hello from main thread");

            for (int i = 0; i < N; i++) {
                  try {
                        threads[i].join();
                  } catch (InterruptedException e) {
                        e.printStackTrace();
                  }
            }
      }
}
```

join method

```
public class MyThread extends Thread {
	public void run() {
		System.out.println("Hello world!");
	}
}
```

```
public class Main {
	public static void main(String[] args) {
		int N = 4;
		MyThread threads[] = new MyThread[N];

		for (int i = 0; i < N; i++) {
			threads[i] = new MyThread();
		}
		for (int i = 0; i < N; i++) {
			threads[i].start();
		}

		System.out.println("Hello from main thread");

		for (int i = 0; i < N; i++) {
			try {
				threads[i].join();
			} catch (InterruptedException e) {
				e.printStackTrace();
			}
		}
	}
}
```

Thread-ul main așteaptă terminarea celorlate thread-uri prin apeluri **join**().

Trebuie să fie într-un bloc try/catch în caz că thread-ul se termină din cauza unei întreruperi.

join method

```
public class MyThread extends Thread {
      public void run() {
            System.out.println("Hello world!");
      }
}
```

```
void join() {
      așteaptă ca un thread să
termine execuția
}
```

```
public class Main {
      public static void main(String[] args) {
            int N = 4;
            MyThread threads[] = new MyThread[N];

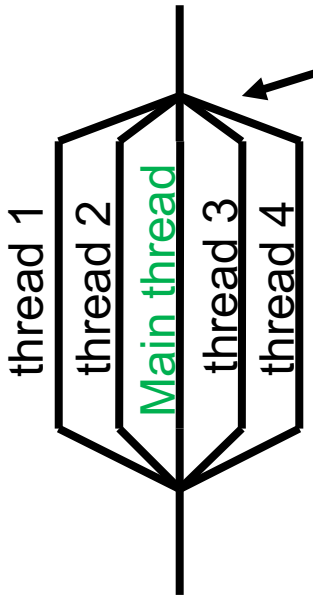
            for (int i = 0; i < N; i++) {
                  threads[i] = new MyThread();
            }
            for (int i = 0; i < N; i++) {
                  threads[i].start();
            }

            System.out.println("Hello from main thread");

            for (int i = 0; i < N; i++) {
                  try {
                        threads[i].join();
                  } catch (InterruptedException e) {
                        e.printStackTrace();
                  }
            }
      }
}
```

thread execution

```
public class MyThread extends Thread {
      public void run() {
            System.out.println("Hello world!");
      }
}
```




```
public class Main {
      public static void main(String[] args) {
            int N = 4;
            MyThread threads[] = new MyThread[N];

            for (int i = 0; i < N; i++) {
                  threads[i] = new MyThread();
            }
            for (int i = 0; i < N; i++) {
                  threads[i].start();
            }
            // Main thread continues execution
            System.out.println("Hello from main thread");

            for (int i = 0; i < N; i++) {
                  try {
                        threads[i].join();
                  } catch (InterruptedException e) {
                        e.printStackTrace();
                  }
            }
      }
}
```

implements instead of extends

```
public class MyThread extends Thread {
      public void run() {
            System.out.println("Hello world!");
      }
}
```

Putem să folosim și implements pentru a crea obiecte ce au punctul de start al unui thread.

De ce? Pentru că în Java un obiect poate moșteni un singur obiect și există cazuri când este necesară extinderea unui alt obiect.

```
public class Main {
      public static void main(String[] args) {
            int N = 4;
            MyThread threads[] = new MyThread[N];

            for (int i = 0; i < N; i++) {
                  threads[i] = new MyThread();
            }
            for (int i = 0; i < N; i++) {
                  threads[i].start();
            }

            System.out.println("Hello from main thread");

            for (int i = 0; i < N; i++) {
                  try {
                        threads[i].join();
                  } catch (InterruptedException e) {
                        e.printStackTrace();
                  }
            }
      }
}
```

implements instead of extends

```
public class MyRunnable implements Runnable {
	public void run() {
		System.out.println("Hello world!");
	}
}
```

```
public class Main {
	public static void main(String[] args) {
		int N = 4;
		Thread threads[] = new Thread[N];

		for (int i = 0; i < N; i++) {
			Runnable aux = new MyRunnable();
			threads[i] = new Thread(aux);
		}
		for (int i = 0; i < N; i++) {
			threads[i].start();
		}

		System.out.println("Hello from main thread");

		for (int i = 0; i < N; i++) {
			try {
				threads[i].join();
			} catch (InterruptedException e) {
				e.printStackTrace();
			}
		}
	}
}
```



APD

Java Concurrency

Sincronizarea firelor de execuție

- **Două situații:**
  - Concurență
  - Cooperare
- **Sincronizarea**
  - asigură ***excluderea mutuală*** – un singur fir poate executa la un moment dat o metodă (secvență de cod) sincronizată: **secțiune critică**
  - Folosește mecanismul de **zăvor** :
    - Fiecare obiect are asociat câte un **zăvor**
    - **synchronized(o)** asigură intrarea în secțiunea critică
    - se poate asocia unei **metode** sau unei **secvențe** de cod
    - pe parcursul execuției secvențelor de cod sincronizate, zăvorul este „închis”

Concurrency problems

```
i = 0;
```

thread 1

```
i = i + 2;
```

thread 2

```
i = i + 2;
```

```
i is 2 or 4
```

Concurrency problems

```
i = 0;
```

thread 1

```
mov eax i;
add eax 2;
mov i eax;
```

thread 2

```
mov eax i;
add eax 2;
mov i eax;
```

```
i is 2 or 4
```

Concurrency problems

```
i = 0;
```

thread 1

```
mov eax i;

add eax 2;

mov i eax;
```

thread 2

```
mov eax i;

add eax 2;
mov i eax;
```

i is **2** or 4

Concurrency problems

```
i = 0;
```

thread 1

```
mov eax i;
add eax 2;
mov i eax;
```

thread 2

```
mov eax i;
add eax 2;
mov i eax;
```

i is 2 or **4**

Solution: synchronized

```
i = 0;
```

thread 1

```
synchronized (object) {
        i = i + 2;
}
```

thread 2

```
synchronized (object) {
        i = i + 2;
}
```

```
i is 4
```

synchronized with object

```
synchronized (object) {
    i = i + 2;
}
```

Echivalent cu:

```
lock (object)
    i = i + 2;
unlock (object)
```

```
object can be any object
object can be this
object can be .class
```

```
public static ThreadSafeSingleton getInstanceUsingDoubleLocking(){
    if(instance == null){
        synchronized (ThreadSafeSingleton.class) {
            if(instance == null){
                instance = new ThreadSafeSingleton();
            }
        }
    }
    return instance;
}
```

synchronized method

Secțiune critică între toate thread-urile care folosesc același **obiect**

```
increment() {
    synchronized (this) {
        i = i + 2;
    }
}
```

```
synchronized increment() {
        i = i + 2;
}
```

Secțiune critică între toate thread-urile care folosesc **clasa *Name***

```
increment() {
    synchronized (Name.class) {
        i = i + 2;
    }
}
```

```
static synchronized increment() {
        i = i + 2;
}
```

Metoda `wait()`

- Permite manevrarea zăvorului asociat cu un obiect
- La apelul metodei `wait()` pentru un obiect ***m*** de către un fir de execuție ***t***:
  - se deblochează zăvorul asociat cu ***m*** și ***t*** este adăugat la un set de thread-uri blocate, *wait set*-ul lui ***m***
  - dacă ***t*** nu deține zăvorul pentru ***m***: `IllegalMonitorStateException`
  - ***t*** va continua execuția doar când va fi scos din *wait set*-ul lui ***m***, prin:
    - o operație **notify() / notifyAll()**
    - expirarea timpului de așteptare
    - o acțiune dependentă de implementare

Notificări

- Metode: `notify()`, `notifyAll()`
- La o notificare apelată din thread-ul ***t*** pentru obiectul ***m***:
  - `notify()`: un thread ***u*** din *wait set*-ul lui ***m*** este scos și repus în execuție
  - `notifyAll()`: toate thread-urile sunt scoase din *wait set*-ul lui ***m*** – dar ***numai unul*** va putea obține zăvorul
  - daca ***t*** nu deține zăvorul pentru ***m***: `IllegalMonitorStateException`

Java – wait()/notify()/notifyAll()

```
this.wait() {
        unlock()
        put_thread_in_wait_state()  //nu mai este pe CPU
        lock()
}
```

Java – wait()/notify()/notifyAll()

Wait funcționează doar dacă ai deja un lock (apelul se face într-o secțiune synchronized)

```
object.wait() {
        unlock()
        put_thread_in_wait_state() //nu mai este pe CPU
        lock()
}
```

Java – wait()/notify()/notifyAll()

```
this.
object.notify() {
        thread = pick_random_thread_waiting_on_object()
        move_from_wait_to_running(thread)
}
```

Java – wait()/notify()/notifyAll()

this.notifyAll()
object.notifyAll()

Pune toate thread-urile care fac wait pe object în starea de running.
Un singur thread va putea lua lock()

Producator-consumator - pseudocod

```
int buf, p=0, c=0;
co process Producer{
        int a[1:n];
    while (p<n){<await p==c>;
          buf = a[p+1];
          p = p+1;}
    }
|| process Consumer{
             int b[1:n];
     while (c<n){<await p>c>;
          b[c+1] = buf;
          c = c+1;}
     }
oc
```

- buf, p, c sunt partajate de cele doua procese
- accesul este exclusiv
- accesul este direct (nu este intermediat de un alt proces)

Clase utile pentru sincronizare în JDK 1.5

- `Semaphore`
- `Mutex`
- `CyclicBarrier`
  - barieră reutilizabilă
  - are ca argument un contor care arată numărul de fire din grup
- `CountDownLatch`
  - similar cu bariera, are un contor, dar decrementarea contorului este separată de așteptarea ajungerii la zero
  - decrementarea semnifică terminarea unor operații
- `Exchanger`
  - *rendez-vous* cu schimb de valori în ambele sensuri între threaduri

Facilități pentru sincronizare de nivel scăzut

- `Lock`
  - generalizare *lock monitor* cu așteptări contorizate, întreruptibile, teste etc.
- `ReentrantLock`
- `Conditions`
  - permit mai multe condiții per lock
- `ReadWriteLock`
  - exploatarea faptului că, la un moment dat, un singur fir modifică datele comune și ceilalți doar citesc
- Variabile atomice:
  - `AtomicInteger`
  - `AtomicLong`
  - `AtomicReference`
  - permit execuția atomică *read-modify-write*

volatile

```
volatile int i;
```

Fiecare scrierea asupra lui i trebuie să ajungă la toate thread-urile

Toate operațiile cu i pot fi mai lente

```
int i;
```

i poate fi ținut în cache (cache posibil ar fi regiștrii CPU)

Toate operațiile cu i pot fi mai rapide

Semaphore

Declararea unui semafor

```
Semaphore semaphore = new Semaphore(4);
```

Permits
(poate fi negativ)

```
try {
        semaphore.acquire();
} catch (InterruptedException e) {
        e.printStackTrace();
}
```

Luarea unui semafor **P()**
(Decremenetează permits)
(Așteaptă dacă permits e 0)

Se eliberează semaforul **V()**
(Crește numărul de permits)

```
semaphore.release();
```

Un semafor inițializat cu 1 este echivalentul unui mutex/lock

Cyclic Barrier

Declararea barierei

Numărul de thread-uri care trebuie să aștepte la barieră ca aceasta să se deschidă

```
CyclicBarrier barrier = new CyclicBarrier(5);
```

Așteaptă până numărul necesar de thread-uri ajung la acest apel.

```
try {
        barrier.await();
} catch (InterruptedException | BrokenBarrierException
e) {
        e.printStackTrace();
}
```

Cyclic Barrier

• O barieră garantează că toate thread-urile execută tot codul de dinainte de barieră înainte de a executa orice linie de cod de după barieră

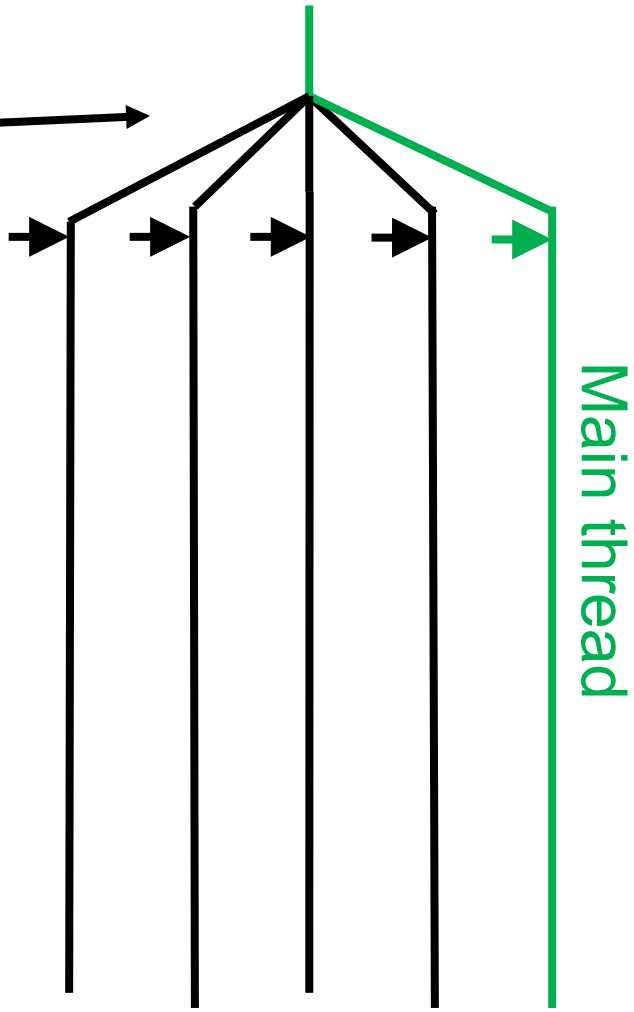
• Barierele clasice nu puteau fi folosite înăuntrul unei iterații
    • După ce toate thread-urile ajung la barieră unul o putea lua înainte și ajunge iar la barieră înainte să se închidă și ar mai trece o dată de ea.

• O barieră **CyclicBarrier** sau **Reentrantă** poate fi refolosită (Este sigură pentru a fi folosită în for/while)

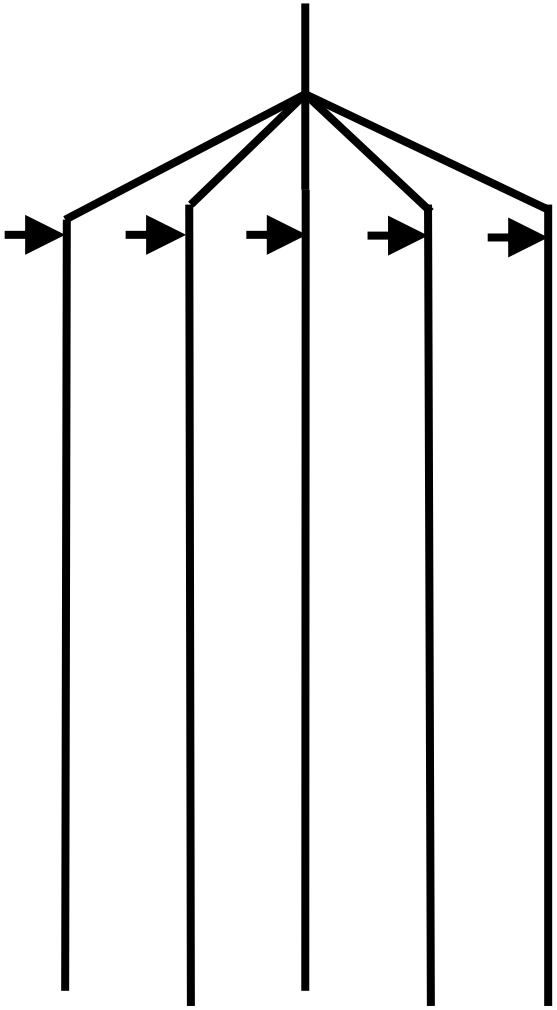
Executor Service – replicated workers

Executor Service

```
ExecutorService tpe =
Executors.newFixedThreadPool(4);
```



Executor Service



```
public class MyRunnable implements Runnable {
	int a;
	ExecutorService tpe;

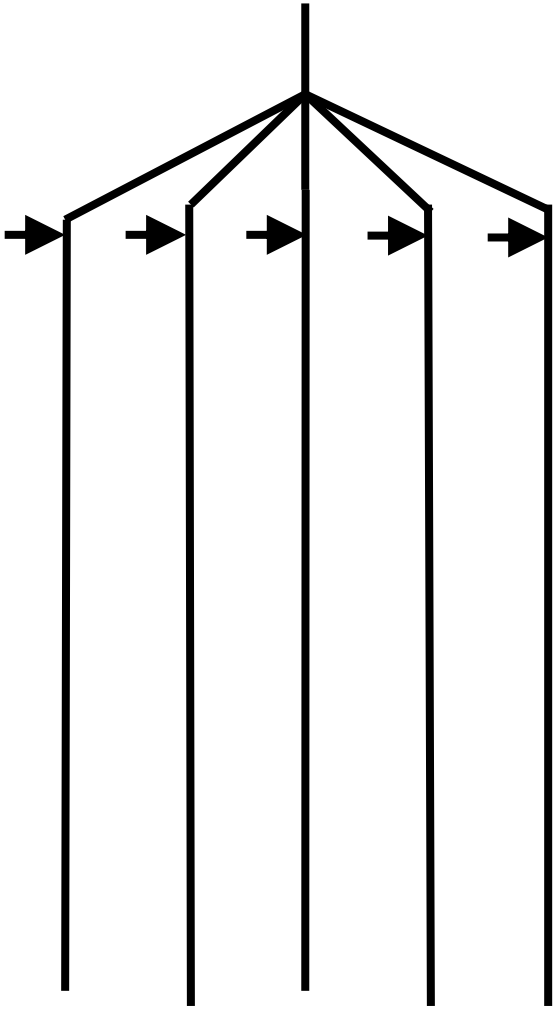
	public MyRunnable(ExecutorService tpe, int a) {
		this.a = a;
		this.tpe = tpe;
	}

	@Override
	public void run() {
		if (a > 10) {
				tpe.shutdown();
				return;
		}
		System.out.println(a);
		tpe.submit(new MyRunnable(tpe, a + 3));
	}
}
```

Executor Service

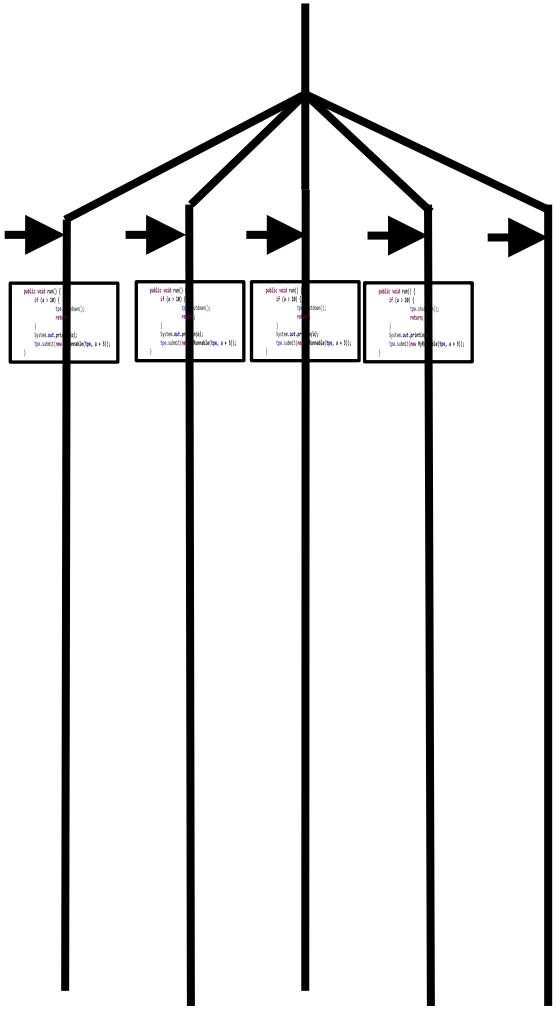
```
tpe.submit(new MyRunnable(tpe, 0));
tpe.submit(new MyRunnable(tpe, 1));
tpe.submit(new MyRunnable(tpe, 2));
……
```





Executor Service

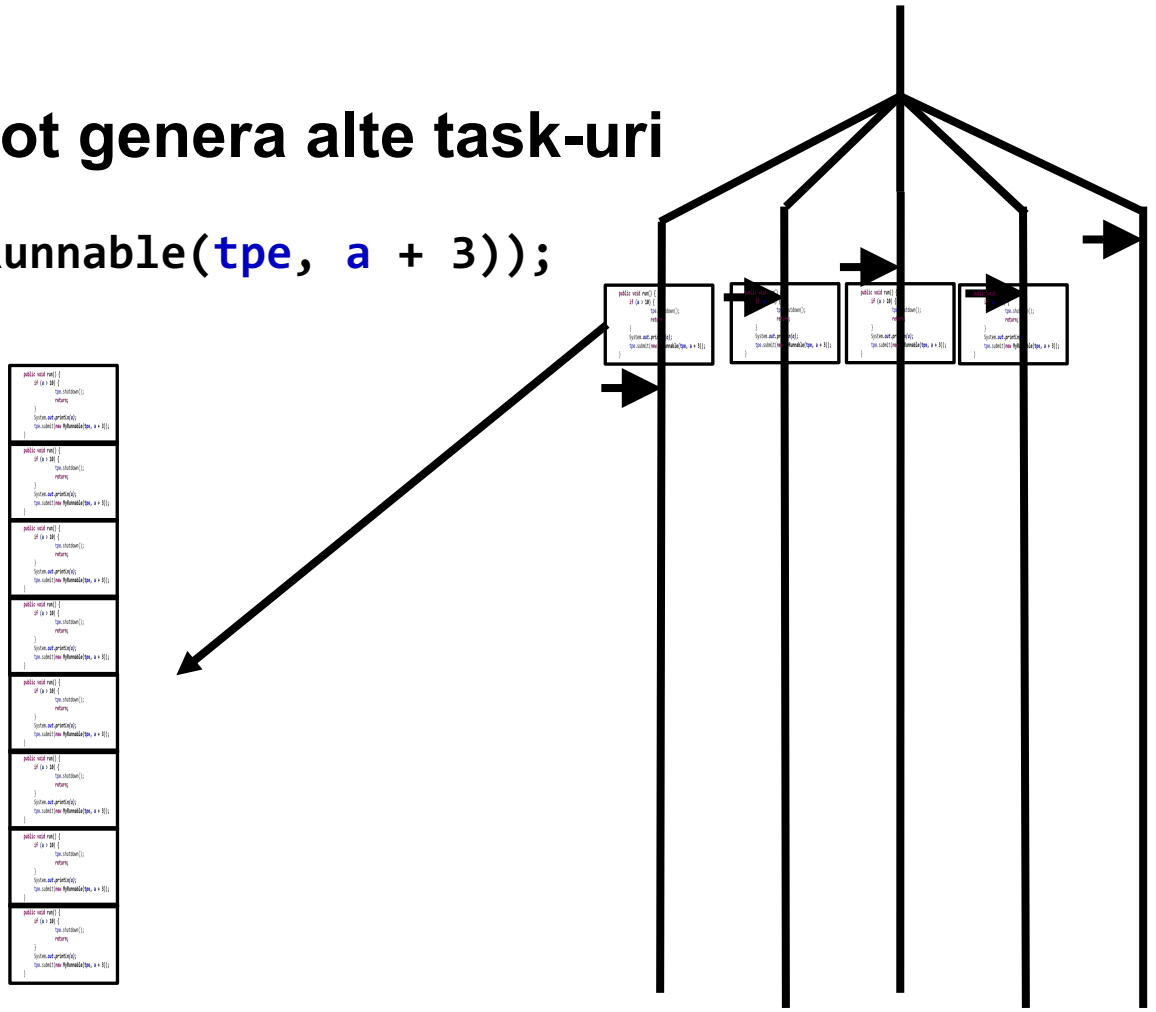




Executor Service

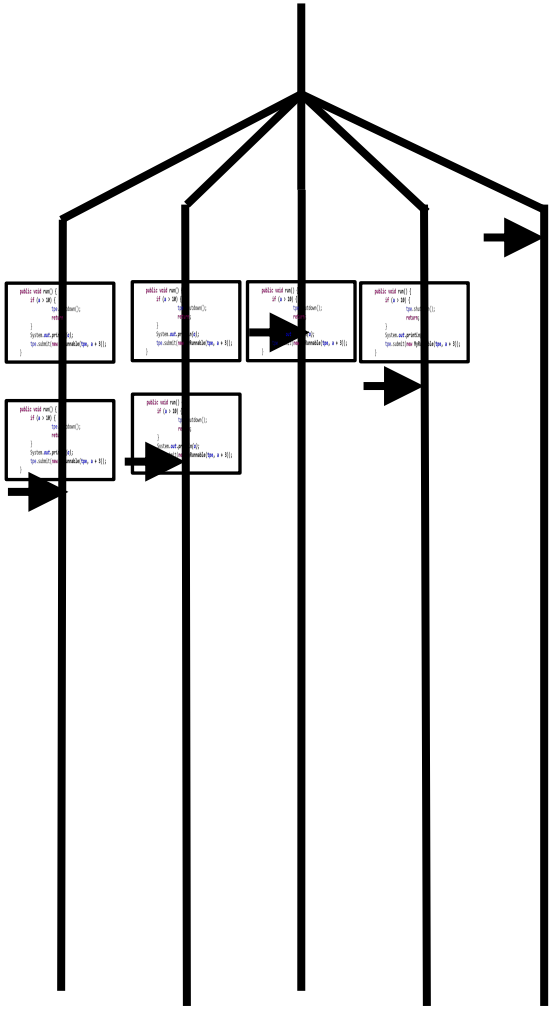
Task-urile pot genera alte task-uri

```
tpe.submit(new MyRunnable(tpe, a + 3));
```



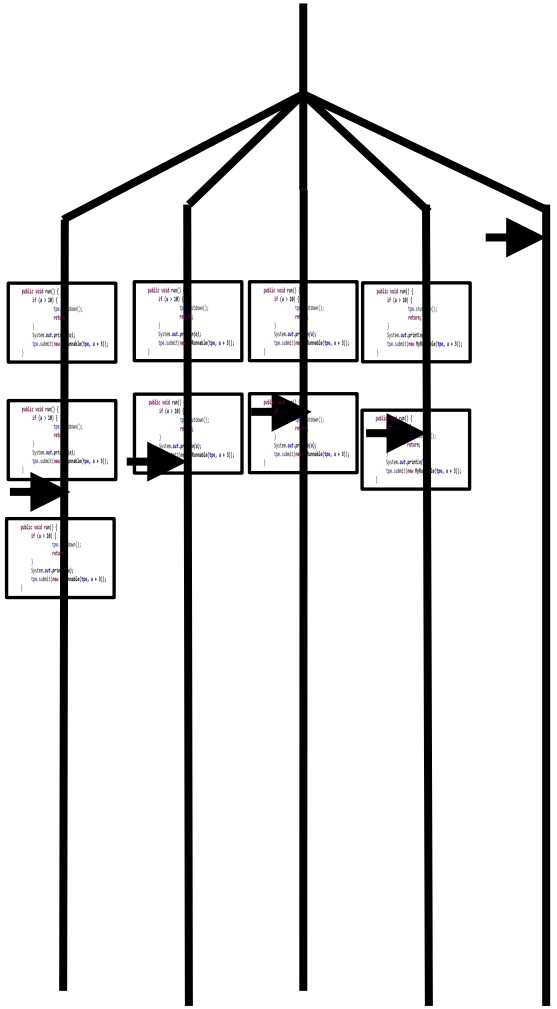
Executor Service





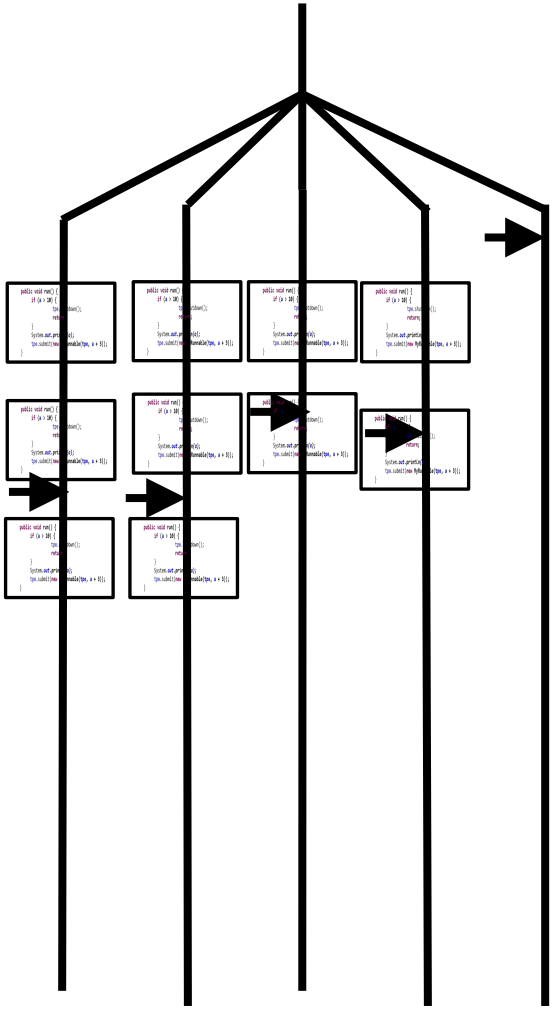
Executor Service





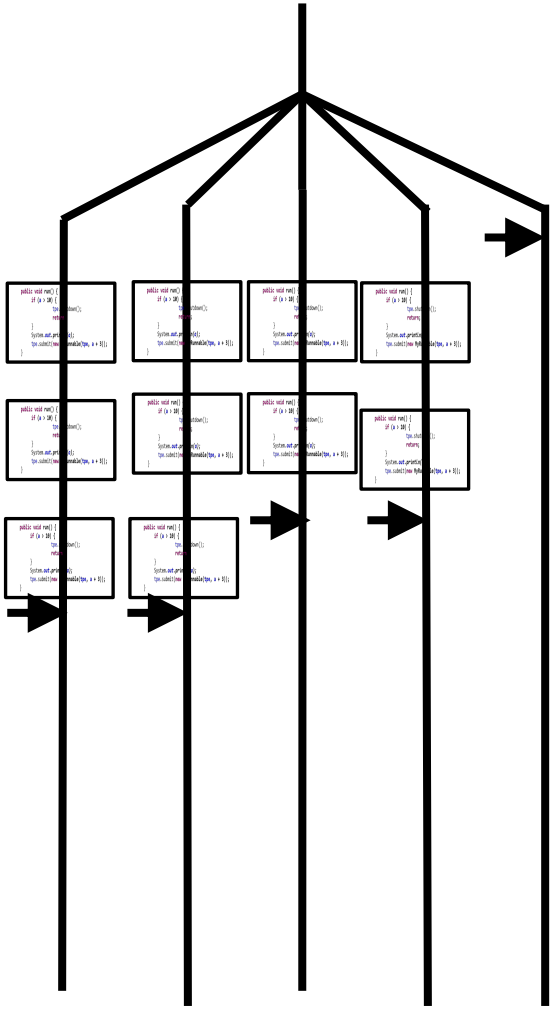
Executor Service





Executor Service

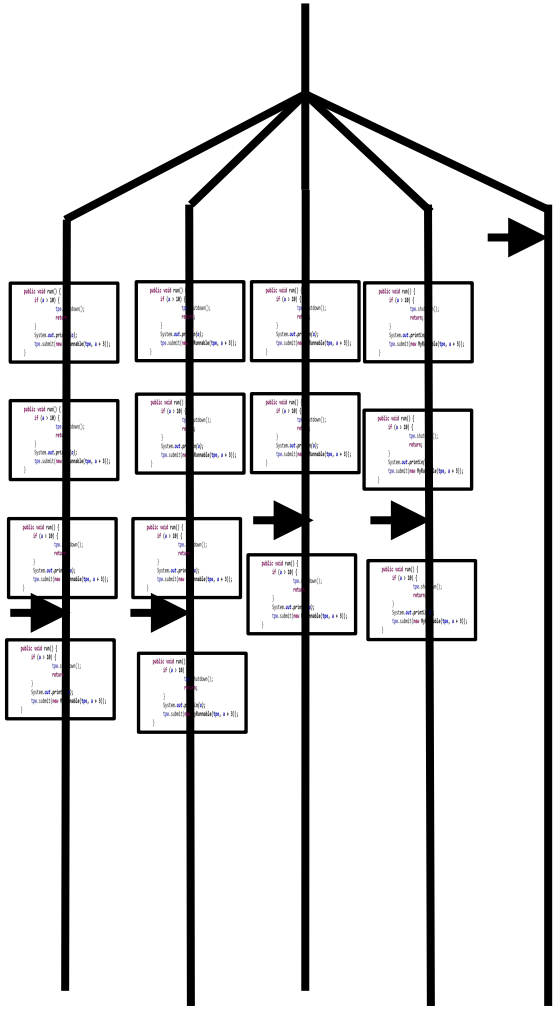




Executor Service

Când oprim thread-urile?

- Depinde de problemă.
- Uneori ne ajunge o singură soluție.
- Alte probleme pot necesita toate soluțiile.

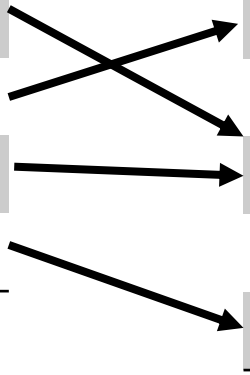


```
tpe.shutdown();
```

Structuri de date

HashMap

Keys	Values
A	11111
DD	2222
C	33333
S	4444



Hash(Keys)	Values
1	2222
2	11111
3	33333
4	
5	4444

Search **O(1)**
Insertion **O(1)**
Deletion **O(1)**

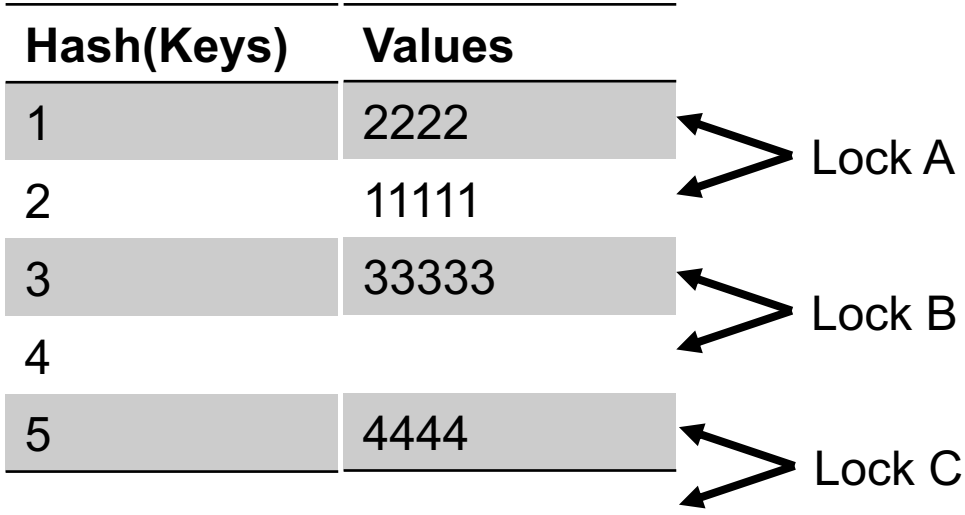
ConcurrentHashMap

- Folosește lock-uri în interiorul tuturor metodelor: **get**(), **put**()
- Asta înseamnă că se poate folosi structura fără cod special pentru sincronizare.
- Are și metode speciale gen **putIfAbsent**()

- Există mai multe astfel de structuri (idem pentru HashTable)

ConcurrentHashMap – privire sub capotă

- Dacă folosim un singur lock pentru accesarea HashMap: 2 thread-uri nu îl vor putea modifica simultan.
- Dacă folosim un lock pentru fiecare element din HashMap: Vom folosi multă memorie și avem nevoie de mecanisme speciale pentru inserție sau ștergere.
- Soluția e folosirea unui număr limitat de lock-uri (de obicei egal cu numărul de thread-uri).



Java – Atomic*

- **AtomicInteger**
- **AtomicLong**
- **AtomicBoolean**
- **AtomicFloat**
- **AtomicReference**

- Problema: 32bit/64bit systems (+ alte arhitecturi)
  - Copierea elementelor de 64 de biți pe procesor de 32 de biți ia mai multe operații iar thread-ul poate fi scos de pe CPU în timpul unei scrieri
  - Un alt thread poate citi doar o parte din variabilă

- **Atomic*** asigură că toate operațiile cu aceea variabilă sunt atomice (nu se pot face citiri în mijloc)
- Oferă și operații mai complexe **getAndSet/getAndIncrement** implementate atomic